

Слайд 5

- веб-сервисы рассчитаны на обслуживание других устройств и сервисов, передают данные при помощи специальных форматов
- веб-приложения ориентированы на взаимодействие непосредственно с пользователем при помощи (графического) интерфейса

Слайд 6

- существует много типов веб-сервисов, но основные:
 - xml-rpc (удаленный вызов процедур)
 - json-rpc (удаленный вызов процедур)
 - soap (обмен структурированной информацией при помощи сообщений)
 - rest (стиль архитектуры распределенных систем; единицы информации или точки управления идентифицируются при помощи url)

Слайд 7

- это не полный список

Слайд 8

- мониторинг ресурсов показал, что сервис не требовательный. подходящие опции - vps, cloud computing, paas, vps
- выбран vps

Слайд 11

- для vps нужна ос, а их много
- поскольку мы используем docker, надо линукс

Слайд 12

- ubuntu и debian оба хороши и стабильны, но первый вариант использует более свежие наборы по
- убунту имеет немного больше удобных и простых инструментов

Слайд 14

- ips: fail2ban, Snort, Tripwire, Suricata
- firewall: iptables, firewalld, ufw
- бд выбрана разработчиком - Firebase Realtime

Слайд 15

- использование root пользователя для логина небезопасно, так что надо создать непривилегированного и открыть доступ только для него
- создание директории

Слайд 16

- оставлять sudo также небезопасно, ибо может использовать злоумышленник
- полное удаление sudo

Слайд 17

- пользователю root надо поменять пароль, чтобы иметь к нему доступ

Слайд 18

- многие скрипты для брутфорс атак используют стандартный порт ssh, так что его надо поменять

Слайд 19

- отключение входа по ssh в root

Слайд 20

- разрешение на вход для созданного аккаунта

Слайд 21

- mosh достаточно установить, так как это не демон

Слайд 22

- хорошей практикой является установка баннеров, уведомляющих о последствиях несанкционированного доступа

Слайд 23

- fail2ban требует только установку и запуск сервиса
- `systemctl enable --now fail2ban`

Слайд 24

- инструкции по установке docker находятся на официальном сайте

Слайд 25

- процесс передачи образа на сервер: сохранение в архив и выгрузка через scp

Слайд 26

- загрузка образа из архива
- запуск контейнеров
- `--name`
- `--port`
- `--restart unless-stopped`

Слайд 27

- в процессе эксплуатации было принято решение проверить лог файлы
- было найдено множество попыток получения несанкционированного доступа к серверу
- попытки: удаленного запуска кода php, вызова командной оболочки, получения переменных среды, запуска различных панелей администрирования, также были попытки эксплуатации уязвимостей сетевого оборудования netgate
- это все делают боты, многие из которых написаны на Go

Слайд 28

- один из китайских адресов помечен в бд как зловредный

Слайд 29

- при попытке запроса по этому адресу открылась страница на китайском языке

Слайд 30

- некоторыми пользователями была замечена реклама на веб-странице Sufflain
- это провайдеры внедряют ее (причем, не только наши, но и американские и китайские)

Слайд 31

- было принято решение подключить HTTPS
- работает с использованием сертификатов
 - есть самоподписанные (не рекомендуется)
 - выдаваемые центрами сертификации
 - раньше были только платные
 - теперь есть что-то типа let's encrypt
 - ручное обновление сертификатов неудобно, поэтому есть программы вроде certbot
- для этого требуется обратный прокси
- выбран nginx, т. к. самый распространенный и имеет много ресурсов от сообщества (гайды, туторы и т. д.)
- надо поставить еще плагин для nginx, чтобы мог работать с certbot

Слайд 32

- запуск certbot с указанием доменных имен, для которых надо настроить https
- потом запросит почту, задаст несколько вопросов
- мы выбрали редирект с 80 на 443

Слайд 33

- в репозиториях ubuntu была только старая версия nginx, которая от 2020 года, так что пришлось обновить для безопасности
- через несколько часов после обновления один из пользователей связался
- была дефолтная страница nginx
- выяснилось, что новая версия из другого репо использовала конфиг из другой директории
- решено переносом конфиг файла

Слайд 34

- использование базы данных как сервиса не надежно, так как в любой момент могут ограничить доступ либо со стороны провайдера, либо со стороны государства
- надежнее иметь базу как часть своей инфраструктуры
- была выбрана CouchDB вместо Firebase Realtime, так как похожий API
- это позволило переписывать не так много кода со стороны разработчика, как могло бы произойти + не так много перенастраивать
- появился доп. элемент - API

Слайд 35

- был развернут сервис, обслуживающий ежедневно небольшое количество пользователей и делающий их жизнь немного удобнее
- сервис был защищен и улучшен
- было изучено много нового и получено большое количество опыта